

SORU 1 :C programlama dilinde 10 elemanlı bir tamsayı dizisi tanımlayınız. Kullanıcıdan dizinin 10

elemanını alınız ve daha sonra bu elemanları ekrana yazdırınız.

Programın:

1. C kodunu yazınız.

Cevap :

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int n = 10;
6      int dizi[n];
7      int i;
8
9      for (i = 0; i < n; i++)
10     {
11         printf("%d. elemani giriniz: ", i + 1);
12         scanf("%d", &dizi[i]);
13     }
14
15     printf("\nGirilen elemanlar:\n");
16
17
18     for (i = 0; i < n; i++)
19     {
20         printf("%d ", dizi[i]);
21     }
22
23     printf("\n");
24     return 0;
25 }
```

```
1. elemani giriniz: 5
2. elemani giriniz: 4
3. elemani giriniz: 3
4. elemani giriniz: 4
5. elemani giriniz: 5
6. elemani giriniz: 6
7. elemani giriniz: 5
8. elemani giriniz: 3
9. elemani giriniz: 6
10. elemani giriniz: 199
```

```
Girilen elemanlar:
5 4 3 4 5 6 5 3 6 199
(base) muhammeteminbas@Emin-MacBook-Air-2 hafta1 %
```

2. $T(n)$ zaman maliyetini hesaplayınız.

Cevap :

Zaman maliyetini ($T(n)$) hesaplariken her bir temel işlemi (atama, karşılaştırma, aritmetik işlem, okuma/yazma) bir maliyet olarak sayabiliriz:

Değişken tanımlamaları `int n=10 ; , int dizi[n] ; , int i ;` Sabit zamanlı işlemlerdir, maliyetlerine c diyelim.

1. Döngü : Elemanları alıyoruz :

`i = 0` ataması: 1 kez çalışır.

`i < n` koşul kontrolü: $n + 1$ kez çalışır.

`i++` artırma işlemi: n kez çalışır.

`printf` ve `scanf` işlemleri: Döngü içinde n kez çalışır.

İlk döngünün toplam maliyeti yaklaşık: $1 + (n + 1) + n + 2n = 4n + 2$

2. Döngü : Elemanları yazdırıyoruz :

`i = 0` ataması: 1 kez çalışır.

`i < n` koşul kontrolü: $n + 1$ kez çalışır.

`i++` artırma işlemi: n kez çalışır.

`printf` işlemi: Döngü içinde n kez çalışır.

İkinci döngünün toplam maliyeti yaklaşık: $1 + (n + 1) + n + n = 3n + 2$

3. $O(n)$ zaman karmaşıklığını bulunuz.

Cevap :

Zaman karmaşıklığında asimptotik (Big-O) gösterim bulunurken $T(n)$ fonksiyonundaki en yüksek dereceli terim dikkate alınır ve katsayılar ile sabitler (algoritmanın büyüme hızını etkilemedikleri için) atılır.

$T(n)=a \cdot n+b$ denkleminde en yüksek dereceli terim n 'dir. Bu nedenle programın zaman karmaşıklığı: $O(n)$ 'dir. (Lineer Zaman / Doğrusal Zaman)

4. Programın $S(n)$ alan karmaşıklığını bulunuz.

Not: n , dizideki eleman sayısını ifade etmektedir.

Cevap :

Alan karmaşıklığı $S(n)$, algoritmanın çalışması için hafızada ayrılan ekstra yerin girdi boyutuyla (n) ilişkisini gösterir.

n elemanlı tam sayı dizisi için: n birim bellek alanı ayrılır. Döngü sayacı i ve sınır değeri n değişkenleri için: Sabit ($1+1$) birim bellek alanı ayrılır.

$S(n) = n + c$ (sabit alanlar) formundadır. Girdi boyutu ne kadarsa bellekte o kadar ekstra yer kapladığı için alan karmaşıklığı asimptotik olarak $O(n)$ 'dir. (Lineer Alan)

SORU 2 :

Kullanıcıdan bir tam sayı alın ve bu sayının bir **Palindrom Sayı** olup olmadığını bulan C programını yazın :

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int sayi, tersSayi = 0, kalan, orijinalSayi;
6
7      printf("Bir tam sayi giriniz: ");
8      scanf("%d", &sayi);
9
10     orijinalSayi = sayi;
11
12     // sayiyi terse cevirme
13     while (sayi != 0)
14     {
15         kalan = sayi % 10;           // sayinin son basamagi
16         tersSayi = tersSayi * 10 + kalan; // Ters sayiya basamagi ekle
17         sayi /= 10;                 // sayinin son basamagini at
18     }
19
20     // karsilastir
21     if (orijinalSayi == tersSayi)
22     {
23         printf("%d bir Palindrom sayidir.\n", orijinalSayi);
24     }
25     else
26     {
27         printf("%d bir Palindrom sayi degildir.\n", orijinalSayi);
28     }
29
30     return 0;
31 }
```

```
Bir tam sayi giriniz: 545
545 bir Palindrom sayidir.
(base) muhammeteminbas@Emin-MacBook-Air-2 hafta1 % cd "/Users/muhammeteminbas/Desktop/VeriYapilari/hafta1/" && gcc ornek2.c -o ornek2 && "/Users/muhammeteminbas/Desktop/VeriYapilari/hafta1/"ornek2
Bir tam sayi giriniz: 567
567 bir Palindrom sayi degildir.
```